MOBL-02: iOS SDK Setup (Swift & SwiftUI)

> **Series:** MOBL — Mobile Monitoring | **Notebook:** 2 of 12 | **Created:** February 2026 | **Last Updated:** 08/04/2026

Overview

This notebook walks through setting up the **Dynatrace Mobile RUM SDK** for iOS applications using **Swift Package Manager (SPM)** or **CocoaPods**. You will learn how to create a mobile app configuration in Dynatrace, install and configure the SDK, instrument both UIKit and SwiftUI views, and verify that session and action data is flowing into your Dynatrace environment.

The Dynatrace iOS SDK provides:
- **Automatic user-action detection** for UIKit view controllers, navigation, and network requests
- **Crash reporting** with symbolicated stack traces
- **SwiftUI instrumentation** via the `.dtAction()` view modifier
- **Manual action and event APIs** for custom business logic

Table of Contents

Prerequisites

Requirement	Details
Xcode	Version 15.0 or later
iOS Deployment Target	iOS 13.0 or later
Dynatrace Environment	SaaS or Managed with Mobile App monitoring enabled
Mobile App Configuration	A mobile app created in Dynatrace (or you will create one in Section 1)
Language	Swift 5.7+
Permissions	`mobile.read`, `entities.read`

1. Creating a Mobile App in Dynatrace

Before integrating the SDK into your iOS project, you need to create a **mobile application configuration** in Dynatrace. This generates two critical values:

- **Application ID** (`DTXApplicationID`) -- uniquely identifies your app in Dynatrace
- **Beacon URL** (`DTXBeaconURL`) -- the endpoint where the SDK sends telemetry data

Steps

1. In your Dynatrace environment, navigate to **Mobile** from the left-hand menu.
2. Click **Create mobile app** (or **Set up mobile monitoring** if this is your first app).
3. Enter a descriptive **Application name** (e.g., `MyCompany iOS App`).
4. Select **iOS** as the platform.
5. Dynatrace generates the **Application ID** and **Beacon URL**. Copy both values -- you will need them in the configuration steps below.
6. Optionally enable **Crash reporting**, **Session replay**, or **User tagging** from the app settings.

> **Tip:** You can find your Application ID and Beacon URL at any time under **Mobile > Your App > Settings > General**.

2. Installing the SDK

The Dynatrace iOS SDK can be installed via **Swift Package Manager (SPM)** or **CocoaPods**. Choose the method that fits your project's dependency management.

Option A: Swift Package Manager (Recommended)

SPM is Apple's native dependency manager and requires no additional tooling.

In Xcode:
1. Go to **File > Add Package Dependencies...**
2. Enter the Dynatrace iOS SPM repository URL.
3. Select the version rule (e.g., **Up to Next Major Version**).
4. Add the `Dynatrace` library to your app target.

```
// Package.swift (if using a Package.swift-based project)
dependencies: [
    .package(url: "https://github.com/Dynatrace/swift-mobile-sdk", from:
"8.0.0")
]
```

Option B: CocoaPods

If your project uses CocoaPods, add the Dynatrace pod to your `Podfile`:

```
# Podfile
platform :ios, '13.0'
use_frameworks!

target 'MyApp' do
  pod 'Dynatrace', '~&gt; 8.x'
end
```

Then run:

```
pod install
```

> **Note:** After installing via CocoaPods, always open the `.xcworkspace` file (not `.xcodeproj`) for subsequent builds.

3. Configuring Info.plist

The simplest way to configure the Dynatrace SDK is through your app's `Info.plist` file. When `DTXAutoStart` is set to `true`, the SDK initializes automatically at app launch.

Add the following keys to your `Info.plist`:

```
&lt;key&gt;DTXApplicationID&lt;/key&gt;
&lt;string&gt;YOUR_APP_ID&lt;/string&gt;
&lt;key&gt;DTXBeaconURL&lt;/key&gt;
&lt;string&gt;YOUR_BEACON_URL&lt;/string&gt;
&lt;key&gt;DTXAutoStart&lt;/key&gt;
&lt;true/&gt;
&lt;key&gt;DTXCrashReportingEnabled&lt;/key&gt;
&lt;true/&gt;
```

Key Reference

Key	Type	Description
`DTXApplicationID`	String	The Application ID from your Dynatrace mobile app configuration
`DTXBeaconURL`	String	The Beacon URL endpoint for telemetry data
`DTXAutoStart`	Boolean	When `true`, the SDK starts automatically at app launch
`DTXCrashReportingEnabled`	Boolean	When `true`, enables crash reporting with symbolicated stack traces
`DTXHybridApplication`	Boolean	Set to `true` if the app uses WKWebView hybrid content

Key	Type	Description
`DTXUserOptIn`	Boolean	When `true`, monitoring only starts after explicit user consent

> **Important:** Replace `YOUR_APP_ID` and `YOUR_BEACON_URL` with the actual values from your Dynatrace mobile app configuration (see Section 1).

4. Auto-Instrumentation (UIKit)

When the SDK is properly configured and `DTXAutoStart` is enabled, the Dynatrace agent automatically instruments standard UIKit components without any code changes. This is the primary advantage of the Dynatrace mobile SDK -- **zero-code instrumentation** for common patterns.

What Gets Auto-Detected

Component	What Is Captured
UIViewController lifecycle	`viewDidAppear`, `viewDidDisappear` -- tracked as user actions with the view controller class name
UITableView / UICollectionView taps	Cell selection events including index path and reuse identifier
UINavigationController	Push/pop navigation transitions and associated view controller names
UITabBarController	Tab selection changes
URLSession network requests	HTTP method, URL, status code, response size, and timing for all requests made through `URLSession`
WKWebView	Web request monitoring within hybrid views (requires `DTXHybridApplication` set to `true`)
App lifecycle	App launch, foreground/background transitions, session start/end

Component	What Is Captured
Crashes	Unhandled exceptions and signal crashes with symbolicated stack traces

How It Works

The SDK uses method swizzling at runtime to intercept UIKit delegate methods and lifecycle callbacks. This means:

- No subclassing or protocol conformance required
- Works with existing `UIViewController` subclasses automatically
- Network monitoring hooks into the `URLSession` delegate chain
- Crash reporting installs signal and exception handlers

> **Note:** Auto-instrumentation covers most common UIKit patterns. For custom gestures, programmatic transitions, or non-standard networking libraries, use the manual action API (covered in **MOBL-03**).

5. SwiftUI Integration

SwiftUI does not use `UIViewController` in the traditional sense, so the auto-instrumentation that works for UIKit requires supplemental instrumentation. Dynatrace provides the `.dtAction()` view modifier for SwiftUI views.

Basic Usage

Apply `.dtAction(name:)` to any view to track it as a user action when it appears on screen:

```
import Dynatrace

struct ContentView: View {
    var body: some View {
        NavigationView {
            List {
                NavigationLink("Products", destination: ProductListView())
                NavigationLink("Cart", destination: CartView())
                NavigationLink("Profile", destination: ProfileView())
            }
            .navigationTitle("Home")
            .dtAction(name: "View Product List")
        }
    }
}
```

Tracking Button Taps

For button taps and other interactions, wrap the action content:

```
import Dynatrace

struct CheckoutView: View {
    var body: some View {
        VStack {
            // ... cart items ...
            Button("Place Order") {
                placeOrder()
            }
            .dtAction(name: "Tap Place Order")
        }
        .dtAction(name: "View Checkout")
    }
}
```

Best Practices for SwiftUI

Practice	Rationale
Apply `.dtAction()` to top-level screen views	Captures screen-level navigation

Practice	Rationale
Use descriptive action names	Makes Dynatrace user session data readable
Add `.dtAction()` to key interaction buttons	Tracks conversion-relevant taps
Avoid applying to every subview	Excess actions create noise in session data

6. Manual Startup Configuration

If you need more control over when and how the SDK starts (for example, to defer initialization until after user consent, or to inject configuration values from a remote config service), you can disable `DTXAutoStart` in `Info.plist` and start the SDK programmatically.

AppDelegate (UIKit Lifecycle)

```
import UIKit
import Dynatrace

class AppDelegate: UIResponder, UIApplicationDelegate {
    func application(
        _ application: UIApplication,
        didFinishLaunchingWithOptions launchOptions:
[UIApplication.LaunchOptionsKey: Any]?
    ) -&gt; Bool {

        Dynatrace.startup(withConfig: [
            "DTXApplicationID": "YOUR_APP_ID",
            "DTXBeaconURL": "YOUR_BEACON_URL",
            "DTXAutoStart": true,
            "DTXCrashReportingEnabled": true
        ])

        return true
    }
}
```

SwiftUI App Lifecycle

For apps using the SwiftUI `App` protocol (no `AppDelegate`):

```
import SwiftUI
import Dynatrace

@main
struct MyApp: App {
    init() {
        Dynatrace.startup(withConfig: [
            "DTXApplicationID": "YOUR_APP_ID",
            "DTXBeaconURL": "YOUR_BEACON_URL",
            "DTXAutoStart": true,
            "DTXCrashReportingEnabled": true
        ])
    }

    var body: some Scene {
        WindowGroup {
            ContentView()
        }
    }
}
```

Deferred Start (User Consent)

If your app requires user opt-in before monitoring:

1. Set `DTXAutoStart` to `false` in `Info.plist` (or omit the key).
2. Call `Dynatrace.startup(withConfig:)` only after the user grants consent.
3. Optionally set `DTXUserOptIn` to `true` for granular consent management.

> **Important:** Replace `YOUR_APP_ID` and `YOUR_BEACON_URL` with the actual values from your Dynatrace mobile app configuration.

7. Verifying Data in Dynatrace

After installing and configuring the SDK, launch your iOS app on a device or simulator and interact with a few screens. Data should begin appearing in Dynatrace within 1-2 minutes.

Where to Check

Location	What to Verify
Mobile > Your App > User Sessions	Sessions are being recorded with user actions
Mobile > Your App > Crashes	Crash reporting is active (trigger a test crash if needed)
Mobile > Your App > Network Requests	HTTP calls from URLSession appear
Notebooks / DQL	Query for mobile entities and actions programmatically

The following DQL queries help confirm that data is arriving from your iOS app.

Find iOS Mobile Applications

This query lists mobile application entities that contain "iOS" in their name or tags, confirming your app is registered in Dynatrace.

```
// Find iOS mobile applications
// PREFERRED -- Smartscape. FRONTEND carries both mobile and web apps, so the
// frontend.type filter is what narrows this to mobile.
smartscapeNodes "FRONTEND"
| filter frontend.type == "mobile"
| filter contains(name, "iOS", caseSensitive: false)
    or contains(toString(tags), "iOS", caseSensitive: false)
| fields name, id, id_classic, tags
| sort name asc

// CORRECTION (07/30/2026): a previous revision claimed
dt.entity.mobile_application had no
// Grail Smartscape equivalent. It does -- FRONTEND filtered on frontend.type
== "mobile"
// (dt.smartscape.frontend). SaaS 1.344 (07/27/2026, staged tenant rollout
from 07/29/2026)
// makes Smartscape the primary surface for the Digital Experience apps;
verify it has
// reached your tenant. The classic table below still works and remains a real
fallback.
// FALLBACK (classic surface -- still functional):
// fetch dt.entity.mobile_application
// | filter contains(toString(entity.name), "iOS") or contains(toString(tags),
"iOS")
// | fields entity.name, id, tags
// | sort entity.name asc
```

Verify Beacon Data Arriving

This query checks for recent mobile user actions received from iOS devices in the last hour. If rows appear, the SDK is successfully sending telemetry.

```
// Check recent mobile user actions (last hour)
fetch bizevents, from:-1h
| filter event.provider == "www.dynatrace.com/mobile"
| filter contains(toString(os.type), "iOS")
| summarize action_count = count(), by:{useraction.name, useraction.type}
| sort action_count desc
| limit 20
```

Look Up App Entity Details

This query retrieves general details for your mobile app entities, including their lifetime and any assigned tags.

```
// iOS app entity details
// PREFERRED -- Smartscape
smartscapeNodes "FRONTEND"
| filter frontend.type == "mobile"
| fields name, id, id_classic, lifetime, tags
| limit 10

// CORRECTION (07/30/2026): a previous revision claimed
dt.entity.mobile_application had no
// Grail Smartscape equivalent. It does -- FRONTEND filtered on frontend.type
== "mobile"
// (dt.smartscape.frontend). SaaS 1.344 (07/27/2026, staged tenant rollout
from 07/29/2026)
// makes Smartscape the primary surface for the Digital Experience apps;
verify it has
// reached your tenant. The classic table below still works and remains a real
fallback.
// FALLBACK (classic surface -- still functional):
// fetch dt.entity.mobile_application
// | fields entity.name, id, lifetime, tags
// | limit 10
```

Summary

In this notebook you learned how to:

- **Create a mobile app configuration** in Dynatrace and obtain the Application ID and Beacon URL
- **Install the Dynatrace iOS SDK** via Swift Package Manager or CocoaPods
- **Configure** `Info.plist` for auto-start, crash reporting, and beacon communication
- **Leverage auto-instrumentation** for UIKit view controllers, navigation, and network requests
- **Instrument SwiftUI views** using the `.dtAction()` view modifier
- **Manually start the SDK** for deferred initialization or user-consent scenarios
- **Verify data flow** using DQL queries against mobile entities and business events

Next Steps

Continue to **MOBL-03** to learn about custom user actions, manual instrumentation APIs, user tagging, and reporting business events from your iOS application.

References

- Dynatrace Mobile Monitoring Documentation
- iOS SDK Integration Guide
- SwiftUI Instrumentation
- Mobile App Configuration Settings
